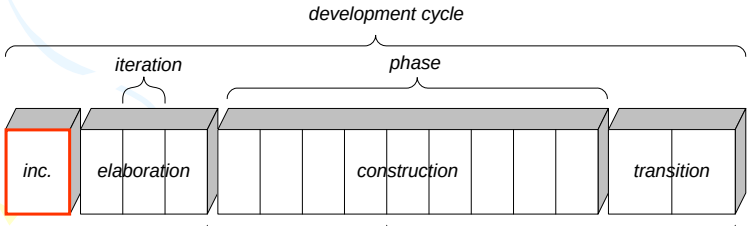
PART II
Inception



1

Chapter 4
Inception is Not the Requirements Phase

2

What is Inception? 1

- What does "inception" mean?
  Answer:
  beginning; start; commencement

全面啟動

What is Inception? 1

- Inception is the initial short step to establish a common vision and basic scope for the project
  - Include analysis of perhaps 10% of the use cases
    - Most requirements analysis occurs during the elaboration phase
  - Analysis of the critical non-functional requirement
  - Creation of a business case, and
  - Preparation of the development environment so that programming can start in the following elaboration phase

What is Inception? 2

- The following kinds of questions are explored
  - What is the vision and business case for this project
  - Feasible?
  - Buy and/or build this system?
  - Rough unreliable range of cost ? Is it $10K, $100K, millions?
    - Obtaining an order-of-magnitude (unreliable) estimate
  - Should we proceed or stop?
  - Do the stakeholders have basic agreement on the vision of the project, and is it worth investing in serious investigation?

5

How Long is Inception?

- Inception phase should be relatively short
  - One or a few weeks long.
    - It is to decide if the project is worth a serious investigation, NOT to do that investigation
    - May include the first requirements workshop, planning for the first iteration, and then quickly moving forward to elaboration

6

Sample Inception Artifacts

Artifact	Comment
Vision and Business Case	Describes the high-level goals and constraints, the business case, and provides an executive summary.
Use-Case Model	Describes the functional requirements. During inception, the names of most use cases will be identified, and perhaps 10% of the use cases will be analyzed in detail.
Supplementary Specification	Describes other requirements, mostly non-functional. During inception, it is useful to have some idea of the key non-functional requirements that have will have a major impact on the architecture.
Glossary	Key domain terminology, and data dictionary.
Risk List & Risk Management Plan	Describes the risks (business, technical, resource, schedule) and ideas for their mitigation or response.
Prototypes and proof-of-concepts	To clarify the vision, and validate technical ideas.
Iteration Plan	Describes what to do in the first elaboration iteration.
Phase Plan & Software Development Plan	Low-precision guess for elaboration phase duration and effort. Tools, people, education, and other resources.
Development Case	A description of the customized UP steps and artifacts for this project. In the UP, one always customizes it for the project.

7

These artifacts are only partially completed in this phase. They will be iteratively refined in subsequent iterations

Chapter 5 Evolutionary Requirements

8

Software Engineering Proverb

The sooner you start, the longer it takes. - Fred Books

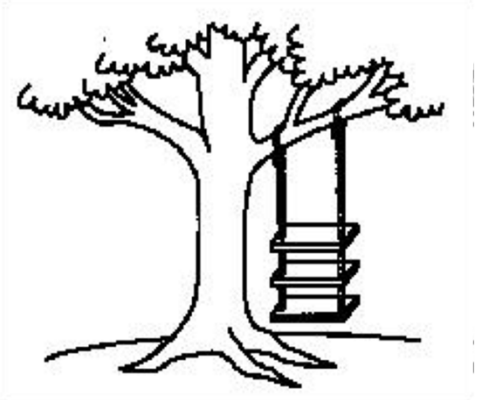
你愈急著開始，花的時間愈長。- Fred Brooks

這是Brooks在1975出版的The Mythical Man Month裡的一句雋語，強調軟體專案的充分準備不致於浪費，**如果你跳過需求蒐集，你設計的東西就非客戶所需，如果你的設計不佳，你將會開發一些無意義的程式**。以今日的軟體發展法來看，這句格言似乎理所當然，而且已歷經三十幾年，其間軟體發展方法也多所改變，但是至今仍然有用，Brooks這句格言是在提醒，從事好的設計仍然是軟體發展的重要過程。

9

Requirements Analysis 1

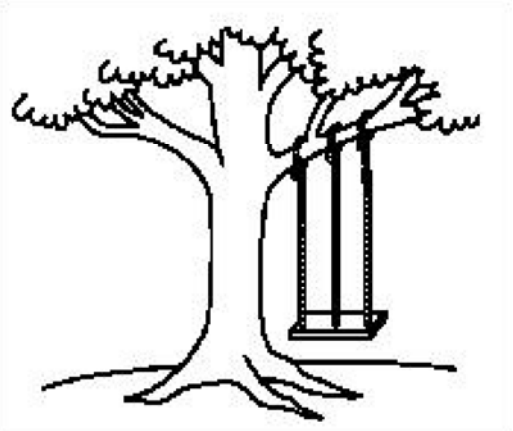
As Marketing requested it (swing).



10

Requirements Analysis 2

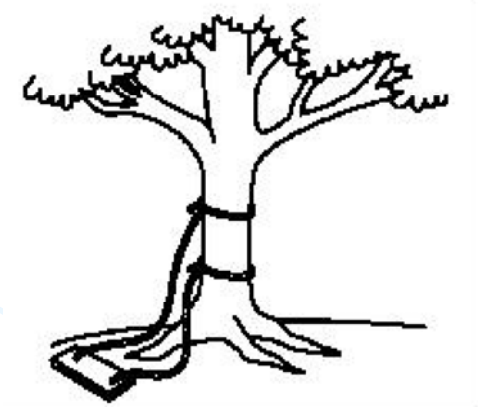
As Sales ordered it.



11

Requirements Analysis 3

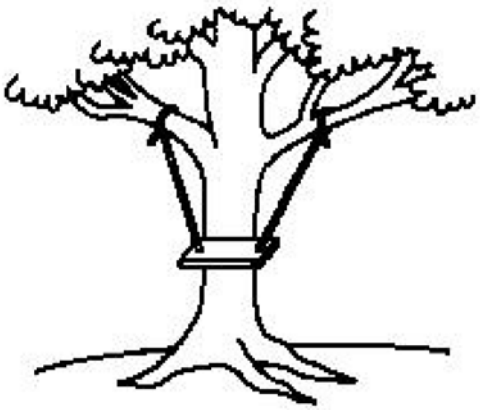
As Engineering designed it.



12

Requirements Analysis 4

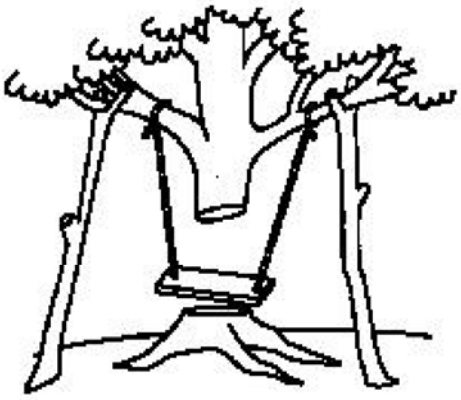
As Production manufactured it.



13

Requirements Analysis 5

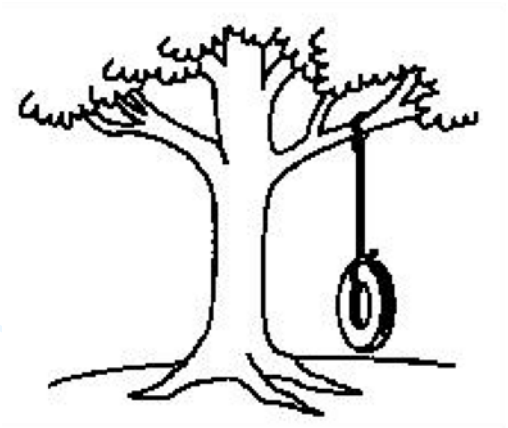
As Maintenance installed it.



14

Requirements Analysis 6

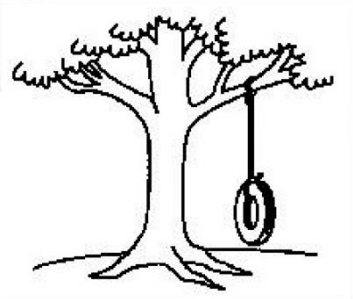
What the customer wanted (next page).



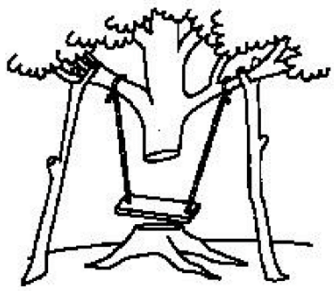
15

Requirements Analysis

- Question: How to avoid distortions when capturing requirements?
  - Customer should be involved; writing use cases.



What the customer wanted.



As Maintenance installed it.

16

What are Requirements?

- Requirements
  - capabilities and conditions to which the system (the project) must conform
  - A prime challenge of requirements analysis is to find, communicate, and remember (write down) what is really needed, in a form that clearly speaks to the client and development team members (unambiguously)
  - UP provides a systematic approach to **finding**, **documenting**, **organizing**, and **tracking** the changing requirements of a system

17

What are Requirements?

- Question: Given two stereo/TV systems of the same price and picture quality, one has the following features and the other does not. Which one would you buy?
  - Support Karaoke/KTV? (Mic, changing keys,…)
  - Support iPhone dock
  - Support Android dock
  - Support Bluetooth
  - Support Miracast/Airplay
  - Support Youtube playback

Next page

18

What are Requirements?

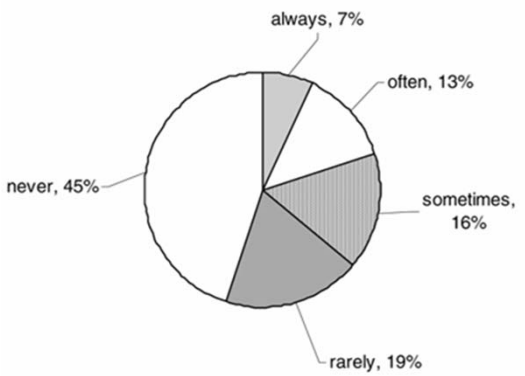
- Question: If you need to pay for each of the following additional features, would you buy all of them?
  - Support Karaoke/KTV? (Mic, changing keys,…)
  - Support iPhone dock
  - Support Android dock
  - Support Bluetooth
  - Support Miracast/Airplay
  - Support Youtube playback

Nice to have ⇔ Must have

19

Actual Use of Waterfall-specified Features

Almost 65% of the waterfall-specified features were of little or no value!



Based on a study of thousands of projects

20

Evolutionary vs. Waterfall Requirements

- Waterfall
  - viewing a software project as similar to predictable mass manufacturing, with low change rates
  - Attempts to freeze or fully define requirements at the start
  - Was unskillful and led to many failures
- Evolutionary
  - Requirements analysis is combined with early design, programming, and testing long before most of the requirements have been analyzed or specified

21

What are Skillful Means to Find Requirements

- Finding the requirements
  - Writing use cases with customers
  - Having requirements workshops that include both developers and customers
  - Focusing groups with proxy customers
  - Having a demo of the results of each iteration to the customers, to solicit feedback

22

The Types and Categories of Requirements 3

- In common usage, requirements are categorized as
  - functional (behavioral)
  - non-functional (everything else)

23

The Types and Categories of Requirements 1

- In UP, requirements are categorized according to the FURPS+ model.
  - Functional: features, capability, security
  - Usability: human factors, help, documentation
  - Reliability: frequency of failure, recoverability, predictability
  - Performance: response times, throughput, accuracy, availability, resource usage.
  - Supportability: adaptability, maintainability, internationalization, configurability

24

The Types and Categories of Requirements 2

- The "+" in FURPS+ indicates sub-factors
  - Implementation: resource limitation, languages and tools, hardware, ..
  - Interface: constraint imposed by interfacing with external systems
  - Operations: system management in its operational setting
  - Packaging: for example, a physical box
  - Legal: licensing and so forth

25

Chapter 6
Use Cases

26

Use Case

- Question: You are outsourcing a software application system. What are the contents of the Software Requirement Specification (SRS) document for your system?

Answer: Use case, Supplementary Specification, Vision Glossary, Business Rules (or Domain Rules)

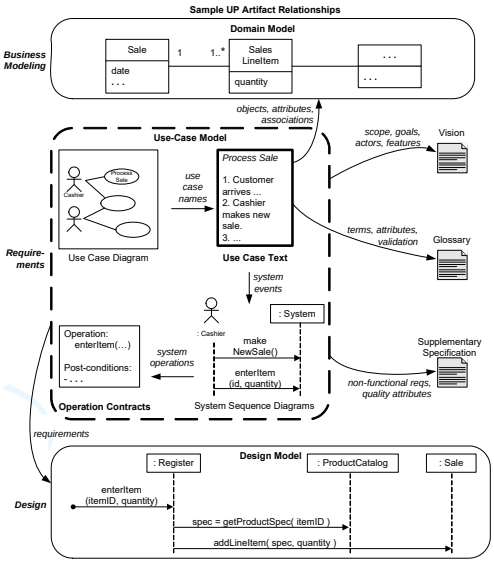
27

Introduction

- The influence of UP artifacts (next page)
  - High-level goals and use case diagrams are input to the creation of the use case text
  - The use cases are input to many other analysis, design, implementation, project management, and test artifacts

28

Sample UP Artifact Influence



Use Case Example

- Informally, use cases are text stories of some actor using a system to meet goals
- An example *brief format* use case
  - Process Sale: A customer arrives at a checkout with items to purchase. The cashier uses the POS system to record each purchased item. The system presents a running total and line-item details. The customer enters payment information, which the system validates and records. The system updates inventory. The customer receives a receipt from the system and then leaves with the items

What are Actors, Scenarios, and Use Cases? 2

- A *casual format* use case with alternate scenarios

Handle Returns

Main Success Scenario: A customer arrives at a checkout with items to return. The cashier uses the POS system to record each returned item …

Alternate Scenarios:

If the customer paid by credit, and the reimbursement transaction to their credit account is rejected, inform the customer and pay them with cash.

If the item identifier is not found in the system, notify the Cashier and suggest manual entry of the identifier code (perhaps it is corrupted).

If the system detects failure to communicate with the external accounting system, …

31

Use Case 1

- Motivation: Why Use Cases?
  - Use case makes it easier for customers to **contribute** to their definition and review
  - Use cases emphasize the user goals and perspective

```
…
for (i = 0; i < 100; i++)
{
    …
}
```

Process Sale: A customer arrives at a checkout with items to purchase. The cashier uses the POS system to record each purchased item. The system presents a running total and line-item details. …

customer

32

Use Case 2

- Question: Are Use Cases Functional Requirements?
  - Answer: Use cases are functional or behavioral requirements that indicate what the system will do. In terms of the FURPS+ requirements types, they emphasize the "F", but can also be used for other types.
- The usage of use case
  - Decide and describe the functional requirements of the system
  - Bring agreement between the customer and software developer
  - Give a clear and consistent description of what the system should do
  - Provide a basis for performing tests that verify the system delivers the functionality stated
  - Trace the functional requirements into actual classes and operations in the system

33

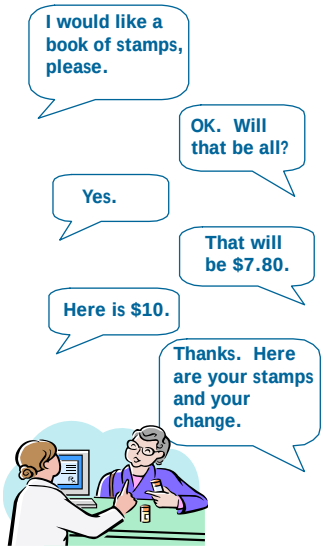
Three Kinds of Actors

- An actor is something with behavior
  - such as a person (identified by role), computer system, or organization; for example, a cashier
- Three kinds of actors
  - **Primary actor** has **user goals fulfilled** through using services. (e.g., the cashier).
  - **Supporting actor** provides a service (e.g., the automated payment authorization service). Often a computer system, but could be an organization or person. The purpose is to **clarify external interfaces** and protocols.
  - **Offstage actor** has an interest in the behavior of the use case, but is not primary or supporting (e.g., a government tax agency).

34

Scenarios 1

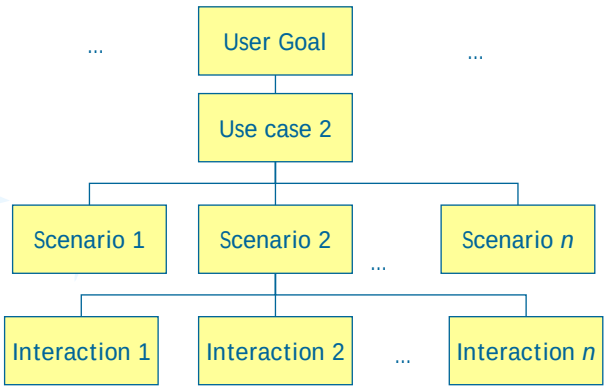
- Scenario
  - Is a specific sequence of actions and interactions between actors and the system; also called a use case instance.
  - It is one particular story of using a system, or one path through the use case.
  - For example, the scenario of successfully purchasing items with cash, or the scenario of failing to purchase items because of a credit payment denial.



35

What are Actors, Scenarios, and Use Cases? 1

- A use case is a collection of related success and failure scenarios that describe an actor using a system to support a goal



36

Use Case 1

- Question: are use cases object-oriented?

  Answer:
  - There is nothing object-oriented about use cases
  - Use cases are a key requirements input to classic OOA/D

37

Three Common Use Case Formats

- Brief (high level)
  - Terse one-paragraph summary, usually of the main success scenario.
  - When? During early requirements analysis, to get a quick sense of subject and scope. May take only a few minutes to create.
- Casual
  - Informal paragraph format. Multiple paragraphs that cover various scenarios.
  - When? As above.
- Fully dressed
  - All steps and variations are written in detail, and there are supporting sections, such as preconditions and success guarantees.
  - When? After many use cases have been identified and written in a brief format, then during the first requirements workshop a few (such as 10%) of the architecturally significant and high-value use cases are written in detail.

38

Fully addressed Use Case Template

Use Case Section	Comment
Use Case Name	Start with a verb.
Scope	The system under design.
Level	"user-goal" or "subfunction"
Primary Actor	Calls on the system to deliver its services.
Stakeholders and Interests	Who cares about this use case, and what do they want?
Preconditions	What must be true on start, and worth telling the reader?
Success Guarantee	What must be true on successful completion, and worth telling the reader.
Main Success Scenario	A typical, unconditional happy path scenario of success.
Extensions	Alternate scenarios of success or failure.
Special Requirements	Related non-functional requirements.
Technology and Data Variations List	Varying I/O methods and data formats.
Frequency of Occurrence	Influences investigation, testing, and timing of implementation.
Miscellaneous	Such as open issues.

39

Fully Use Case Example 1

Use Case UC1: Process Sale (16 pages)

Scope: NextGen POS application
Level: user goal
Primary Actor: Cashier
Stakeholders and Interests:

- Cashier: Wants accurate, fast entry, and no payment errors, as cash drawer shortages are deducted from his/her salary.
- Salesperson: Wants sales commissions updated.
- Customer: Wants purchase and fast service with minimal effort. Wants easily visible display of entered items and prices. Wants proof of purchase to support returns.
- Company: Wants to accurately record transactions and satisfy customer interests. Wants to ensure that Payment Authorization Service payment receivables are recorded. Wants some fault tolerance to allow sales capture even if server components (e.g., remote credit validation) are unavailable. Wants automatic and fast update of accounting and inventory.

40

Fully Use Case Example 2

Use Case UC1: Process Sale

- Manager: Wants to be able to quickly perform override operations, and easily debug Cashier problems.
- Government Tax Agencies: Want to collect tax from every sale. May be multiple agencies, such as national, state, and county.
- Payment Authorization Service: Wants to receive digital authorization requests in the correct format and protocol. Wants to accurately account for their payables to the store.

Preconditions: Cashier is identified and authenticated.

Success Guarantee (or Postconditions): Sale is saved. Tax is correctly calculated. Accounting and Inventory are updated. Commissions recorded. Receipt is generated. Payment authorization approvals are recorded.

41

Fully Use Case Example 3

Use Case UC1: Process Sale

Main Success Scenario (or Basic Flow)

1. Customer arrives at POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total. Price calculated from a set of price rules.

Cashier repeats steps 3-4 until indicates done.

5. System presents total with taxes calculated.
6. Cashier tells Customer the total, and asks for payment.
7. Customer pays and System handles payment.
8. System logs completed sale and sends sale and payment information to the external Accounting system (for accounting and commissions) and Inventory system (to update inventory).
9. System presents receipt.
10. Customer leaves with receipt and goods (if any).

42

Fully Use Case Example 4

Use Case UC1: Process Sale

Extensions (or Alternative Flows)

*a. At any time, Manager requests an override operation:

1. System enters Manager-authorized mode.
2. Manager or Cashier performs one Manager-mode operation. e.g., cash balance change, resume a suspended sale on another register, void a sale, etc.
3. System reverts to Cashier-authorized mode.

*b. At any time, System fails: To support recovery and correct accounting, ensure all transaction sensitive state and events can be recovered from any step of the scenario.

1. Cashier restarts System, logs in, and requests recovery of prior state.
2. System reconstructs prior state.
   - 2a. System detects anomalies preventing recovery:
     1. System signals error to the Cashier, records the error, and enters a clean state.
     2. Cashier starts a new sale.

43

Fully Use Case Example 5

Use Case UC1: Process Sale

Extensions (or Alternative Flows) (cont'd)

1a. Customer or Manager indicate to resume a suspended sale.

1. Cashier performs resume operation, and enters the ID to retrieve the sale.
2. System displays the state of the resumed sale, with subtotal.
   - 2a. Sale not found.
     1. System signals error to the Cashier.
     2. Cashier probably starts new sale and re-enters all items.
3. Cashier continues with sale (probably entering more items or handling payment).

2-4a. Customer tells Cashier they have a tax-exempt status (e.g., seniors, native peoples)

1. Cashier verifies, and then enters tax-exempt status code.
2. System records status (which it will use during tax calculations)

44

Fully Use Case Example 6

Use Case UC1: Process Sale

Extensions (or Alternative Flows) (cont'd)

- 3a. Invalid item ID (not found in system):
  - 1. System signals error and rejects entry.
  - 2. Cashier responds to the error:
    - 2a. There is a human-readable item ID (e.g., a numeric UPC):
      - 1.Cashier manually enters the item ID.
      - 2.System displays description and price.
      - 2a. Invalid item ID: System signals error. Cashier tries alternate method
    - 2b. There is no item ID, but there is a price on the tag:
      - 1.Cashier asks Manager to perform an override operation.
      - 2.Managers performs override.
      - 3.Cashier indicates manual price entry, enters price, and requests standard taxation for this amount (because there is no product information, the tax engine can't otherwise deduce how to tax it)
    - 2c. Cashier performs Find Product Help to obtain true item ID and price.
    - 2d. Otherwise, Cashier asks an employee for the true item ID or price, and does either manual ID or manual price entry (see above).

45

Fully Use Case Example 7

Use Case UC1: Process Sale

Extensions (or Alternative Flows) (cont'd)

- 3b. There are multiple of same item category and tracking unique item identity not important (e.g., 5 packages of veggie-burgers):
  - 1.Cashier can enter item category identifier and the quantity.
- 3c. Item requires manual category and price entry (such as flowers or cards with a price on them):
  - 1.Cashier enters special manual category code, plus the price.
- 3-6a: Customer asks Cashier to remove (i.e., void) an item from the purchase: This is only legal if the item value is less than the void limit for Cashiers, otherwise a Manager override is needed.
  - 1.Cashier enters item identifier for removal from sale.
  - 2.System removes item and displays updated running total.
    - 2a. Item price exceeds void limit for Cashiers:
      - 1.System signals error, and suggests Manager override.
      - 2.Cashier requests Manager override, gets it, and repeats operation.

46

Fully Use Case Example 8

Use Case UC1: Process Sale

Extensions (or Alternative Flows) (cont'd)

3-6b. Customer tells Cashier to cancel sale:

1. Cashier cancels sale on System.

3-6c. Cashier suspends the sale:

1. System records sale so that it is available for retrieval on any POS register.
2. System presents a "suspend receipt" that includes the line items, and a sale ID used to retrieve and resume the sale.

4a. The system supplied item price is not wanted (e.g., Customer complained about something and is offered a lower price):

1. Cashier requests approval from Manager.
2. Manager performs override operation.
3. Cashier enters manual override price.
4. System presents new price.

47

Fully Use Case Example 9

Use Case UC1: Process Sale

Extensions (or Alternative Flows) (cont'd)

5a. System detects failure to communicate with external tax calculation system service:

1. System restarts the service on the POS node, and continues.
   - 1a. System detects that the service does not restart.
     1. System signals error.
     2. Cashier may manually calculate and enter the tax, or cancel the sale.

5b. Customer says they are eligible for a discount (e.g., employee, preferred customer):

1. Cashier signals discount request.
2. Cashier enters Customer identification.
3. System presents discount total, based on discount rules.

48

Fully Use Case Example 10

Use Case UC1: Process Sale

Extensions (or Alternative Flows) (cont'd)

5c. Customer says they have credit in their account, to apply to the sale:

1. Cashier signals credit request.
2. Cashier enters Customer identification.
3. Systems applies credit up to price=0, and reduces remaining credit.

6a. Customer says they intended to pay by cash but don't have enough cash:

1. Cashier asks for alternate payment method.
   - 1a. Customer tells Cashier to cancel sale. Cashier cancels sale on System.

7a. Paying by cash:

1. Cashier enters the cash amount tendered.
2. System presents the balance due, and releases the cash drawer.
3. Cashier deposits cash tendered and returns balance in cash to Customer.
4. System records the cash payment.

49

Fully Use Case Example 11

Use Case UC1: Process Sale

Extensions (or Alternative Flows) (cont'd)

7b. Paying by credit:

1. Customer enters their credit account information.
2. System displays their payment for verification.
3. Cashier confirms.
   - 3a. Cashier cancels payment step:
     1. System reverts to "item entry" mode.
4. System sends payment authorization request to an external Payment Authorization Service System, and requests payment approval.
   - 4a. System detects failure to collaborate with external system:
     1. System signals error to Cashier.
     2. Cashier asks Customer for alternate payment.

50

Fully Use Case Example 12

Use Case UC1: Process Sale

Extensions (or Alternative Flows) (cont'd)

5. System receives payment approval, signals approval to Cashier, and releases cash drawer (to insert signed credit payment receipt).
   - 5a. System receives payment denial:
     1. System signals denial to Cashier.
     2. Cashier asks Customer for alternate payment.
   - 5b. Timeout waiting for response.
     1. System signals timeout to Cashier.
     2. Cashier may try again, or ask Customer for alternate payment.
6. System records the credit payment, which includes the payment approval.
7. System presents credit payment signature input mechanism.
8. Cashier asks Customer for a credit payment signature. Customer enters signature.
9. If signature on paper receipt, Cashier places receipt in cash drawer and closes it.

51

Fully Use Case Example 13

Use Case UC1: Process Sale

Extensions (or Alternative Flows) (cont'd)

- 7c. Paying by check...
- 7d. Paying by debit...
- 7e. Cashier cancels payment step:
  1. System reverts to "item entry" mode.
- 7f. Customer presents coupons:
  1. Before handling payment, Cashier records each coupon and System reduces price as appropriate. System records the used coupons for accounting reasons.
     - 1a. Coupon entered is not for any purchased item:
       1. System signals error to Cashier.
- 9a. There are product rebates:
  1. System presents the rebate forms and rebate receipts for each item with a rebate.
- 9b. Customer requests gift receipt (no prices visible):
  1. Cashier requests gift receipt and System presents it.
- 9c. Printer out of paper.
  1. If System can detect the fault, will signal the problem.
  2. Cashier replaces paper.
  3. Cashier requests another receipt.

52

Fully Use Case Example 14

Use Case UC1: Process Sale

Special Requirements

- Touch screen UI on a large flat panel monitor. Text must be visible from 1 meter.
- Credit authorization response within 30 seconds 90% of the time.
- Somehow, we want robust recovery when access to remote services such the inventory system is failing.
- Language internationalization on the text displayed.
- Pluggable business rules to be insertable at steps 3 and 7.
- ...

53

Fully Use Case Example 15

Use Case UC1: Process Sale

Technology and Data Variations List:

*a. Manager override entered by swiping an override card through a card reader, or entering an authorization code via the keyboard.

3a. Item identifier entered by bar code laser scanner (if bar code is present) or keyboard.

3b. Item identifier may be any UPC, EAN, JAN, or SKU coding scheme.

7a. Credit account information entered by card reader or keyboard.

7b. Credit payment signature captured on paper receipt. But within two years, we predict many customers will want digital signature capture.

Frequency of Occurrence: Could be nearly continuous.

54

Fully Use Case Example 16

Use Case UC1: Process Sale

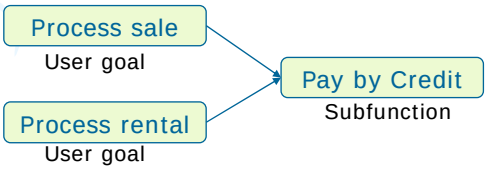
Open Issues:

- What are the tax law variations?
- Explore the remote service recovery issue.
- What customization is needed for different businesses?
- Must a cashier take their cash drawer when they log out?
- Can the customer directly use the card reader, or does the cashier have to do it?

55

What do the Sections in Use Case Template Mean? 1

- **Scope**:
  - bounds the system (or systems) under design
- **Level**:
  - A user-goal level use case
    - The common kind that describe the scenarios to fulfill the goals of a primary actor to get work done
  - A subfunction-level use case
    - Describes sub-steps required to support a user goal, and is usually created to factor out duplicate sub-steps shared by several regular use case (e.g., Pay by Credit)

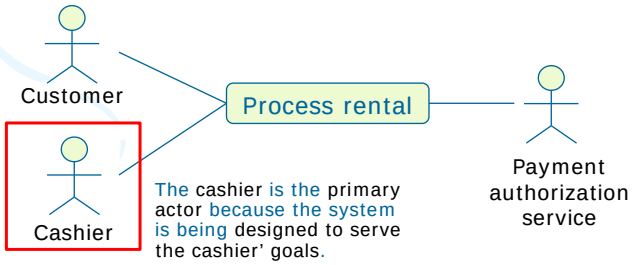


56

What do the Sections in Use Case Template Mean? 2

- **Primary Actor**:
  - The principal actor that calls upon system services to fulfill a goal

Question: who is the primary actor of the Process rental use case?



57

What do the Sections in Use Case Template Mean? 2

- **Stakeholders**（利害關係人）**and Interests**（利益）**List**：—**Important!**
  - **Project stakeholders**
    - Entities that have an interest in the project.
    - Stakeholders may be inside or outside an organization
    - Examples: Project leader, Project team members, Senior management, Project customer, Resource Managers, Line Managers, Product user group, Project testers, Subcontractors to the project, Consultants to the project
  - Start writing the stakeholders and their interests reminds us the more detailed responsibilities of the system.
  - The stakeholder interest viewpoint provides a thorough and methodical procedure for discovering and recording all the required behaviors

58

What do the Sections in Use Case Template Mean? 3

- **Preconditions and Success Guarantees** (Postconditions):
  - Don't bother with a precondition or success guarantee unless you are stating something non-obvious and noteworthy, to help the reader gain insight
  - Preconditions: state what *must always* be true before a scenario is begun in the use case.
  - Success guarantees (or postconditions): state what must be true after successful completion of the use case either the main success scenario or some alternate path

Process sale example

Preconditions: Cashier is identified and authenticated.
Success Guarantee (or Postconditions): Sale is saved. Tax is correctly calculated. Accounting and Inventory are updated. Commissions recorded. Receipt is generated. Payment authorization approvals are recorded.

59

What do the Sections in Use Case Template Mean? 4

- **Main Success Scenario**:
  - Also called the "happy path" scenario, "Basic Flow", or "Typical Flow"
    - A typical success path that satisfies the interests of the stakeholders
  - Records the steps of:
    - An interaction between actors
    - A validation (usually by the system)
    - A state change by the system (e.g., recording or modifying something)
  - Always capitalize the actors' names (e.g., Cashier)
  - Guideline: Defer all conditional and branching statements to the Extensions section.

Question: why deferring all conditional and branching statements to the Extensions section?

Answer: more comprehensible and extendible (although having conditions and branching in the main success scenario is not wrong or illegal).

60

What do the Sections in Use Case Template Mean? 5

- **Extensions** (or Alternate Flows):
  - Normally comprise the majority of the text
  - Indicate all the other scenarios or branches, both success and failure
  - An extension has two parts: the condition and the handling
    - One-step condition example: 3a
    - Multiple-steps condition example: 2-4a
    - Any-steps condition example: *a, *b
    - The letters "a", "b", ... stands for case a, case b, etc.
    - At the end of extension handling, the scenario **merges back with the main success scenario**, unless the extension indicates otherwise (such as by halting the system)

61

Guideline: Write in an Essential UI-Free Style 2

- Question: which of the following writings is better? (A) or (B)?

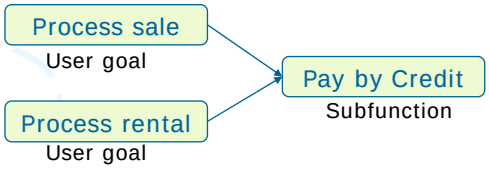
(A)	... 5a. System detects failure to communicate with external tax calculation system ...
(B)	... 5a. External tax calculation system not working ...

Guideline: When possible, write the condition as something that can be **detected by** the system or an actor.

62

What do the Sections in Use Case Template Mean? 7

- Sometimes, a particular extension point is quite complex, (e.g., "paying by credit" extension).
  - This can be a motivation to express the extension as a separate subfunction-level use case
    - For example, the story Find Product Help is a distinct use case that is sometimes performed while within Process Sale
    - In Cockburn notation, performing this second use case is shown with **underlining**



63

What do the Sections in Use Case Template Mean? 8

- **Special Requirements**:
  - If a **non-functional requirement**, quality attribute, or constraint relates specifically to a use case, record it with the use case.
  - These include qualities such as performance, reliability, and usability, and design constraints (often in I/O devices) that have been mandated or considered likely

64

What do the Sections in Use Case Template Mean? 9

- **Technology and Data Variations List**:
  - Often there are technical variations in how something must be done, but not what, and it is noteworthy to record this in the use case.
  - A common example is a technical constraint imposed by a stakeholder regarding input or output technologies (e.g., support credit account input using a card reader and the keyboard)
  - It is also necessary to understand variations in data schemes, such as using UPCs or EANs for item identifiers, encoded in bar code symbology

65

Use cases give context

- Question: Why not use a feature list to capture the requirements? Is the following feature list a good requirement document? (next page)

Feature List

1. The system can record sales
2. The cashier can enter items to the system
3. The system can show running total
4. The system can handle payments
5. The system can produce receipts
6. The system can compute taxes

66

Use cases give context

- Good context information → better understanding

The Romance of the Condor Heroes

楊過吐出一口鮮血 (遇襲?)

Yan spits out some blood (ambushed?)

楊過坐正，吐出一口鮮血 (比武後?)

Yan sits up and spits out some blood (after a fight?)

楊過坐正，張嘴吐出一口鮮血 (不緊急?)

Yan sits up, opens his mouth, and spits out some blood (not urget?)

67

Use cases give context

楊過坐正，張嘴吐出一口鮮血，鬆了一口氣 (吐血是好事?)

Yan sits up, opens his mouth, and spits out some blood. He feels relieved. (blood spitting is good?)

經過幾番折騰，楊過坐正，張嘴吐出一口鮮血，鬆了一口氣 (故意的？)

After several rounds, Yan sits up, opens his mouth, and spits out some blood. He feels relieved. (on purpose?)

經過幾番折騰，牙醫終於拔出智齒，楊過坐正，張嘴吐出一口鮮血，鬆了一口氣 (!@#$%^)

After several rounds, the dentist finally takes out Yan's wisdom tooth. Yan sits up, opens his mouth, and spits out some blood. He feels relieved. (!@#$%^?)

68

Use cases give context

- Good context information → better understanding
- Difficult to understand → little or no context information
- Use cases give context and keep readers from getting lost in a sea of requirements (next page)

69

Use cases give context

Main Success Scenario

1. Customer arrives at POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total. Price calculated from a set of price rules.

Cashier repeats steps 3-4 until indicates done.

5. System presents total with taxes calculated.
6. Cashier tells Customer the total, and asks for payment.
7. Customer pays and System handles payment.
8. System logs completed sale …
9. System presents receipt.

Feature List

1. The system can record sales
2. The cashier can enter items to the system
3. The system can show running total
4. The system can handle payments
5. The system can produce receipts
6. The system can compute taxes

No context information

70

Guideline: Write in an Essential UI-Free Style 2

- Question: which of the following writings related to login is better? (A) or (B)?

(A)	... Administrator enters ID and password in dialog box (see Picture 3). System authenticates Administrator. ...
(B)	... Administrator identifies self. System authenticates identity. ...

71

Guideline: Write in an Essential UI-Free Style 1

- During a requirements workshop, the cashier may say one of his goals is to "log in."
  - The cashier was probably thinking of a GUI, dialog box, user ID, and password. This is a mechanism to achieve a goal, rather than the goal itself.
  - This root-goal discovery process can open up the vision to new and improved solutions (e.g. RFID card, fingerprint, face detection...)
- Essential Style Writing
  - During **early requirements work**, "keep the user interface out, focus on intent."
  - They remain free of technology and mechanism details, especially those related to the UI

72

Guideline: Write in an Essential UI-Free Style 2

- **Concrete style**
  - user interface decisions are embedded in the use case text. (avoid during early requirements work). For example:
    - Administrator enters ID and password in dialog box (see Picture 3).
    - System authenticates Administrator.
    - System displays the "edit users" window (see Picture 4).
    - ...
- **Essential Style**
  - focus on intent, not the UI
    - Administrator identifies self.
    - System authenticates identity.

Guideline: Write use cases in an essential style; keep the user interface out and focus on actor intent

73

Guideline: Write in an Essential UI-Free Style 2

- Question: which of the following writings related to login is better? (A) or (B)?

(A)	... System authenticates ... ...
(B)	... The System authenticates ... ...

74

Guideline: Write Terse Use Cases

- Guideline: Write Terse Use Cases
  - Delete "noise" words.
  - Even small changes add up,
    - such as "System authenticates..."
    - rather than "~~The~~ System authenticates..."

75

Guideline: Write Black-Box Use Cases

- Question: which of the follow writings is the best? (A), (B), or (C)?

(A)	... The system records the sale. ...
(B)	... The system writes the sale to a database. ...
(C)	... The system generates a SQL INSERT statement for the sale. ...

76

Guideline: Write Black-Box Use Cases

- Guideline: Write Black-Box Use Cases
  - Do not describe the internal workings of the system, its components, or design.
  - Specify what the system must do (the behavior or functional requirements) without deciding how it will do it (the design).
  - The definition of "analysis" versus "design" is "what" versus "how."

Black-box style	Not Black-box style
The system records the sale.	The system writes the sale to a database. … or (even worse): The system generates a SQL INSERT statement for the sale…

77

Guideline: How to Find Use Cases 1

- Guideline: How to Find Use Cases?
  1. Choose the system boundary.
  2. Identify the primary actors
     - that have goals fulfilled through using services of the system.
  3. Identify the goals for each primary actor.
  4. Define use cases that satisfy user goals; name them according to their goal.
     - Usually, user-goal level use cases will be one-to-one with user goals

Guideline: Take an Actor and Actor-Goal Perspective

Focusing on the actors of a system, asking about their goals and typical situations. Focus on understanding what the actor considers a valuable result.

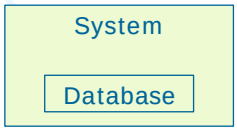
78

Guideline: How to Find Use Cases 1

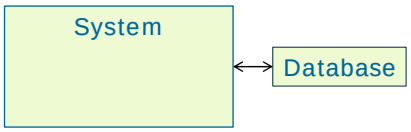
- **Step 1. Choose the system boundary.**
  - Is it just a software application, the hardware and application as a unit, that plus a person using it, or an entire organization?

- Question: Is database inside your system boundary?

(a)



(b)



Answer: It depends.

79

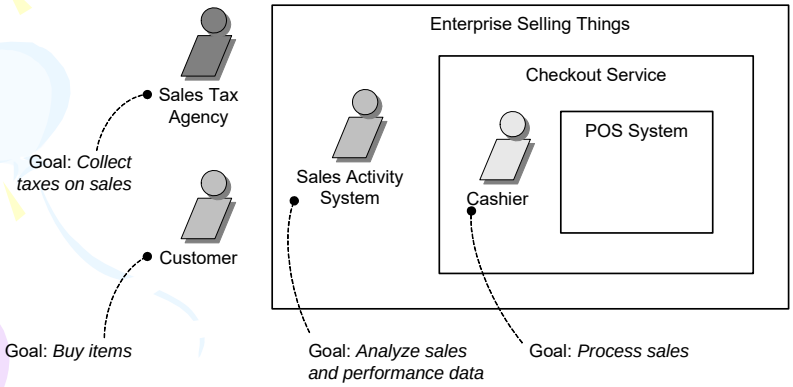
Guideline: How to Find Use Cases 2

- **Step 2. Identify the primary actors**
  - that have goals fulfilled through using services of the system.
- Guideline: Brainstorm the primary actors first, as this sets up the framework for further investigation

80

Who Is The Primary Actor?

- Primary actors and goals at different system boundaries



Who Is The Primary Actor?

- NextGen POS
  - The cashier is the primary actor because the system is being designed to serve the cashier' goals.
  - The customer is an actor, but not a primary actor
- Ticket-buying Website
  - Customer is the primary actor because the system is designed for the customer to use directly
- Ticket-buying with a phone agent
  - When a customer calls in, the agent is simply a proxy of the customer
  - The system is NOT designed to meet the goals of the agent
  - Showing the customer as the primary actor is correct.

Guideline: How to Find Use Cases 3

- **Step 3. Identify the goals for each primary actor**
- Questions to Help Find Actors and Goals?
  - Who starts and stops the system?
  - Who does system administration?
  - Who does user and security management?
  - Is "time" an actor because the system does something in response to a time event?
  - Is there a monitoring process that restarts the system if it fails?
  - Who evaluates system activity or performance?
  - How are software updates handled? Push or pull update?
  - Who evaluates logs? Are they remotely retrieved?
  - In addition to human primary actors, are there any external software or robotic systems that call upon services of the system?
  - Who gets notified when there are errors or failures?

83

Why Ask About Actor Goals Rather Than Use Cases?

- Question: when interviewing with a customer/actor (e.g., a staff of academic affair) in a requirements workshop, which of the following questions should you ask? (A) or (B)

✗	(A) What do you do?
○	(B) What are your goals?

84

Why Ask About Actor Goals Rather Than Use Cases?

- Find actors and their goals, and create solutions that produce a result of value
- In a requirements workshop, we could ask either:
  - "What do you do?"
    - Roughly a task-oriented question or
  - "What are your goals whose results have measurable value?"
    - It opens up the vision for new and improved solutions, focus on adding business value, and get to the heart of what the stakeholders want from the system

85

Guideline: How to Find Use Cases 5

- Step 4: Define Use Cases
  - Define one use case for each user goal.
  - Name the use case similar to the user goal
    - For example, Goal: process a sale; Use Case: Process Sale.
  - Start the name of use cases with a **verb**.
  - A common exception to one use case per goal is to collapse CRUD (create, retrieve, update, delete) separate goals into one CRUD use case, idiomatically called Manage <X>.
    - For example, the goals "edit user," "delete user," and so forth are all satisfied by the Manage Users use case.

86

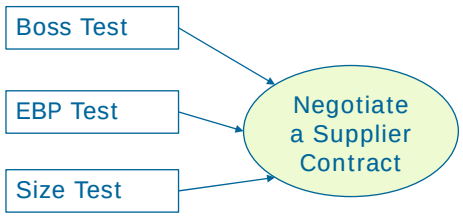
Guideline: What Tests Can Help Find Useful Use Cases 1

- Question: Which of the followings are valid use cases (for application requirements analysis)?
  - Negotiate a Supplier Contract
  - Handle Returns
  - Log In
  - Move Piece on Game Board (Monopoly)

87

Guideline: What Tests Can Help Find Useful Use Cases 1

- What is a useful level to express use cases for application requirements analysis? The rules of thumb:
  - The Boss Test
  - The EBP Test
  - The Size Test



88

Guideline: What Tests Can Help Find Useful Use Cases 2

- The Boss Test
  - Your boss asks, "What have you been doing all day?" You reply: "Logging in!" Is your boss happy?
  - If yes, the use case passes Boss Test
  - If not, the use case fails Boss Test
  - That doesn't mean to always ignore boss-test-failing use cases.
    - For example, user authentication may fail the boss test, but may be important and difficult

89

Guideline: What Tests Can Help Find Useful Use Cases 3

- The EBP Test
  - Elementary Business Process (EBP)
    - A task performed by one person in one place at one time, in response to a business event, which adds measurable business value and leaves the data in a consistent state, e.g., Approve Credit or Price Order
  - Does the use case reflect an EBP?
    - If yes, the use case passes EBP Test
    - If not, the use case fails EBP Test

90

Guideline: What Tests Can Help Find Useful Use Cases 4

- The Size Test
  - A use case typically contains many steps (often require 3~10 pages of text in the fully dressed format)
    - A use case is very seldom a single step/action
    - A common mistake is to define just a single step as a use case, such as defining a use case called Enter an Item ID.
    - You can see a hint of the error by its small size
  - Is the use case big enough (having many steps)?
    - If yes, the use case passes Size Test.
    - If not, the use case fails Size test.

91

Guideline: What Tests Can Help Find Useful Use Cases 5

- Applying the Tests

	Boss Test	EBP Test	Size Test
Negotiate a Supplier Contract	OK	Not OK: much broader and longer than an EBP. Could be modeled as a business use case, rather than a system use case	NA
Handle Returns	OK	OK	OK
Log In	Not OK	OK	NA
Move Piece on Game Board	OK	OK	Not OK

92

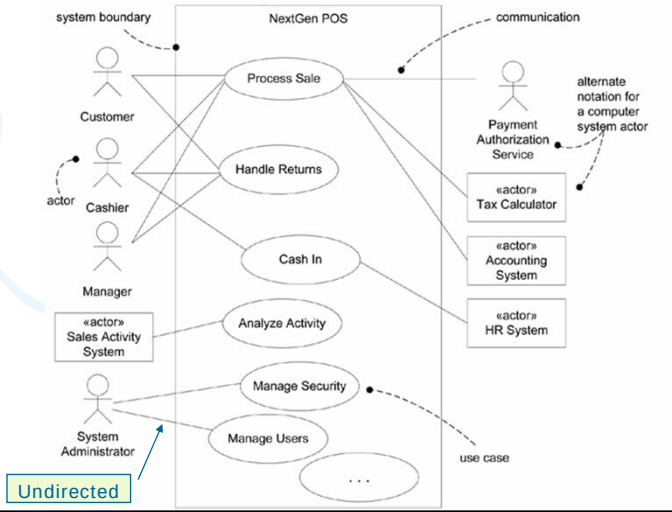
Guideline: What Tests Can Help Find Useful Use Cases 5

- Most use cases should satisfy the tests, but exceptions are common
  - Authenticate User is may be complex enough to warrant careful analysis (e.g., single sign-on").

93

Applying UML: Use Case Diagrams 1

- UML Use Case Diagram



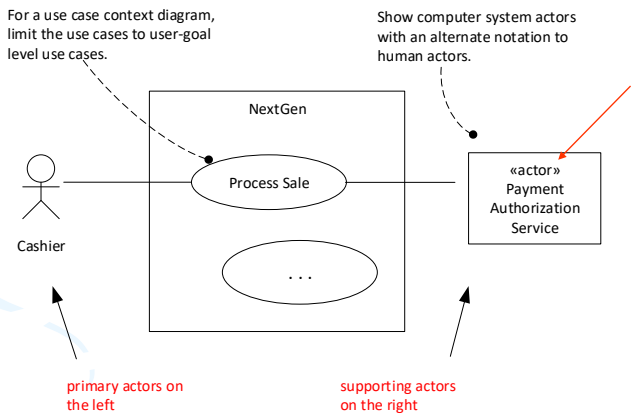
94

Applying UML: Use Case Diagrams 1

- UML Use Case Diagram
  - Illustrates the names of use cases and actors, and the relationships between them
  - Use case diagram makes a good context diagram, summarizing the behavior of a system and its actors.
  - Showing the boundary of a system, what lies outside of it, and how it gets used.
- Guideline: Downplay Diagramming, Keep it Short and Simple.
  - If an organization is spending many hours/days working on a use case diagram and discussing use case relationships, rather than focusing on writing text, effort has been misplaced.

95

Applying UML: Use Case Diagrams 3

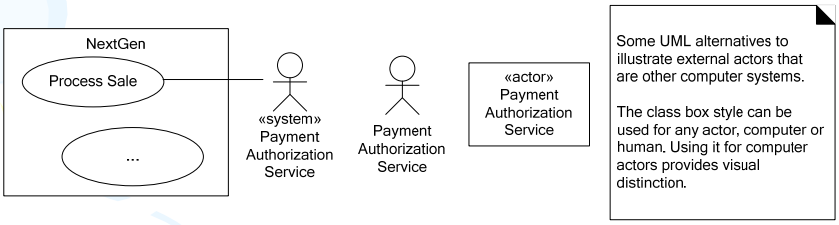


Notation suggestions

96

Applying UML: Use Case Diagrams 4

To clarify, some prefer to highlight external computer system actors with an alternate notation



Alternate actor notation

97

Motivation: Other Benefits of Use Cases 1

- Use cases replace detailed, low-level function lists with use cases

ID	Feature
FEAT1.9	The system shall accept entry of item identifiers.
...	...
FEAT2.4	The system shall log credit payments to the accounts receivable system.

- However, High-Level System Feature Lists are acceptable
  - A terse, high-level feature list, called **system features**, added to a Vision document can usefully summarize system functionality.
  - Summary of System Features, for **POS.**
    - sales capture
    - payment authorization (credit, debit, check)
    - system administration for users, security, code and constants tables, and so on
    - ...

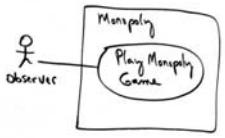
98

Motivation: Other Benefits of Use Cases 2

- Question: when are detailed feature lists appropriate rather than use cases?
  - Sometimes use cases do not really fit; some applications cry out for a feature-driven viewpoint (e.g., application servers, database products, and other middleware or back-end systems)

99

Example: Monopoly Game



Use Case UC1: Play Monopoly Game
Scope: Monopoly application
Level: user goal
Primary Actor: Observer
Stakeholders and Interests:
Observer: Wants to easily observe the output of the game simulation.
Main Success Scenario:
1. Observer requests new game initialization, enters number of players.
2. Observer starts play.
3. System displays game trace for next player move (see domain rules, and "game trace" in glossary for trace details).

Repeat step 3 until a winner or Observer cancels.

Extensions:
*a. At any time, System fails:
(To support recovery, System logs after each completed move)
1. Observer restarts System.
2. System detects prior failure, reconstructs state, and prompts to continue.
3. Observer chooses to continue (from last completed player turn).

Special Requirements:
Provide both graphical and text trace modes.

100

Work With Use Cases in Iterative Methods

- UP encourages **use-case driven development**.
  - Use cases are an important part of iterative planning.
    - The work of an iteration is in part defined by choosing some use case scenarios, or entire use cases.
  - Use cases are a key input to estimation.
  - Use-case realizations drive the design.
    - The team designs collaborating objects and subsystems in order to perform or realize the use cases.
  - Use cases often influence the organization of user manuals.
  - Functional or system testing corresponds to the scenarios of use cases.
  - UI "wizards" or shortcuts may be created for the most common scenarios of important use cases to ease common tasks.

101

Evolve Use Cases Across the Iterations 1

Discipline	Artifact	Incep 1 week	Elab 1 4 weeks
Requirements	Use-Case Model	2-day requirements workshop. Most use cases identified by name, and summarized in a short paragraph. Pick 10% from the high-level list to analyze and write in detail. This 10% will be the most architecturally important, risky, and high-business value.	Near the end of this iteration, host a 2-day requirements workshop. Obtain insight and feedback from the implementation work, then complete 30% of the use cases in detail.

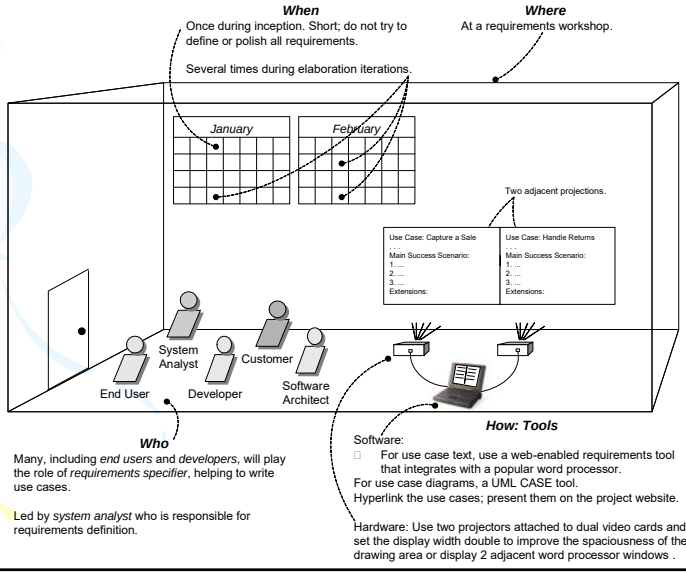
102

Evolve Use Cases Across the Iterations 2

Elab 2 **4 weeks**	Elab 3 3 weeks	Elab 4 3 weeks
Near the end of this iteration, host a 2-day requirements workshop. Obtain insight and feedback from the implementation work, then complete 50% of the use cases in detail.	Repeat, complete 70% of all use cases in detail.	Repeat with the goal of 80~90% of the use cases clarified and written in detail. Only a small portion of these have been built in elaboration; the remainder are done in construction.

103

Process And Setting Context For Writing Use Cases



104

Case Study: Use Cases in the NextGen Inception Phase

Fully Dressed	Casual	Brief
Process Sale Handle Returns	Process Rental Analyze Sales Activity Manage Security ...	Cash In Cash Out Manage Users Start Up Shut Down Manage System Tables ...

105

Sample UP Artifacts and Timing

Discipline	Artifact	Incep.	Elab.	Const.
	Iteration	I1	E1..En	C1..Cn
Business Modeling	Domain Model		s	
Requirements	Use-Case Model	s	r	
	Vision	s	r	
	Supplementary Specification	s	r	
	Glossary	s	r	
Design	Design Model		s	r
	SW Architecture Document		s	

106

Chapter 7 Other Requirements

107

Other Requirement Artifacts

- Supplementary Specification (non-functional requirements)
  - Captures and identifies other kinds of requirements, such as reports, documentation, packaging, supportability, licensing, ..
- Vision
  - Summarizes the "vision" of the project an executive summary. It serves to tersely communicate the big ideas.
- Glossary ← **Required in the homework assignments**
  - Captures terms and definitions; it can also play the role of a data dictionary.
- Business Rules (or Domain Rules)
  - Capture long-living and spanning rules or policies, such as tax laws, that transcend one particular application.
- Guideline: These Artifacts should be at the Project Website

108

Waterfall ⇔ Iterative

109

Reliable Specifications: An Oxymoron?

- Writing a Vision and Supplementary Specification is worthwhile as an exercise in clarifying a first approximation of what is wanted
  - the motivation for the product, and as a repository for the big ideas.
  - But they are not nor is any requirements artifact — a reliable specification
- Treat written requirements lightly, start programming early, and continually — ideally, daily — engage users and tests for feedback

110

POS Supplementary Specification 1

- Question: which of the followings are non-functional requirements of an online shopping system?
  - (A) A user can collect coupon and get discount.
  - (B) The system allows 100 users to shop online at the same time.
  - (C) The system keeps shopping records for all users.
  - (D) The server is Linux based.
  - (E) When completing an order, the user should receive an email.
  - (F) The system should be virus scanned every day.
  - (G) The system should collect user habits and show related ADs.
  - (H) Credit card authorization should be completed in 10 seconds.
  - (I) The system should support at least 10000 user accounts.
  - (J) The system (server) should be backed up every day.
  - (K) A user can shop with either browser, Android, or iOS.

Performance: (B), (H), (I); Compatibility: (D), (K); Security/reliability: (F), (J)

111

POS Supplementary Specification (3 pages)

- Revision History

See the text book for a full supplementary specification document

Version	Date	Description	Author
Inception draft	Jan 10, 2031	First draft. To be refined primarily during elaboration.	Craig Larman

- Introduction
  - This document is the repository of all NextGen POS requirements not captured in the use cases.
- Functionality
  - Logging and Error Handling
    - Log all errors to persistent storage…
- Usability
  - Human Factors: The customer will be able to see a large-monitor display of the POS…
- Reliability
  - Recoverability: if there is failure…

112

POS Supplementary Specification 2

- Performance:
  - As mentioned under human factors, buyers want to complete sales processing very quickly…
- Supportability
  - Adaptability:
    - Different customers have unique business rule …
- Implementation Constraints
  - NextGen leadership insists on a Java technologies solution…
- Purchased Components
  - Tax calculator. Must support pluggable calculators for different countries.
- Free Open Source Components
  - … maximizing the use of free Java technology open source …
- Interfaces
  - Noteworthy Hardware and Interfaces
    - Touch screen monitor …

113

POS Supplementary Specification 6

- Application-Specific Domain (Business) Rules
  - (See the separate Business Rules document for general rules.)

ID	Rule	Changeability	Source
RULE 1	Purchaser discount rules. Examples: Employee 20% off. Preferred Customer 10% off. Senior15% off.	High. Each retailer uses different rules.	Retailer policy.
…	…	…	…

- Legal Issues
  - … All tax rules must, by law, be applied during sales…
- Information in Domains of Interest
  - Pricing
    - In addition to the pricing rules described in the domain rules section, …

114

Commentary: Supplementary Specification 1

- Supplementary specification
  - Captures other requirements, information, and constraints not easily captured in the use cases or glossary
  - Including system-wide "URPS+" (usability, reliability, performance, supportability, and more) quality attributes or non-functional requirements.
  - Initially, non-functional requirements specific to a use case can be briefly written in the special requirements section of the use case
  - Later, use-case-specific non-functional requirements should be moved to the Supplementary Specification, to keep all non-functional requirements in one place, and not duplicated.

115

POS Supplementary Specification

Question: Can you write a good requirement documentation, including both functional and non-function requirements?

Non-functional requirements tend to be ignored. Why?

Experience

116

Commentary: Vision

- When a new member joins the project,
  - "Welcome! Please go read the 7-page Vision at the project website."
- It is also useful to have an executive summary that briefly describes the project, as a context for the major players to establish a common vision of the project.
- It should summarize some of the information in the Use-Case Model and Supplementary Specification

117

NextGen Example: (Partial) Vision

(3 pages)

See the text book for a full vision document

- Revision History

Version	Date	Description	Author
Inception draft	**Jan 10, 2031**	**First draft. To be refined primarily during elaboration.**	**Craig Larman**

- Introduction
  - We envision a next generation fault-tolerant point-of-sale (POS) application, NextGen POS, with the flexibility to support varying customer business rules, multiple terminal and user interface mechanisms, and integration with multiple third-party supporting systems.
- Positioning
  - ...Existing POS products are not adaptable to the customer's business, ..., they do not scale .... And, none can work in either on-line or off-line mode, dynamically adapting depending on failures. None easily integrate with many third-party systems...

118

NextGen Example: (Partial) Vision

- Stakeholder Descriptions
  - ...
- Product Overview
  - Product Perspective
    - The NextGen POS will usually reside in stores; if mobile terminals are used, they will be in close proximity to the store network, either inside or close outside. It will provide services to users, and collaborate with other systems, as indicated in Figure Vision-1.

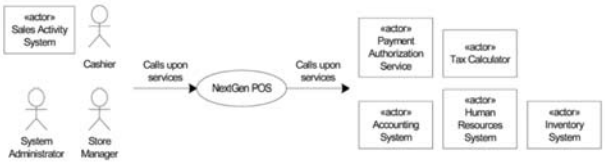


Figure vision- 1. NextGen POS system context diagram

119

NextGen Example: (Partial) Vision

- Summary of System Features
  - sales capture
  - payment authorization (credit, debit, check)
  - system administration for users, security, code and constants tables, and so forth.
  - automatic offline sales processing when external components fail
  - real-time transactions, based on industry standards, with third-party systems, including inventory, accounting, human resources, tax calculators, and payment authorization services
  - definition and execution of customized "pluggable" business rules at fixed, common points in the processing scenarios
  - ...
- Other Requirements and Constraints
  - Including design constraints, usability, reliability, performance, supportability, design constraints, documentation, packaging, and so forth: See the Supplementary Specification and use cases.

120

Commentary: Vision 3

- Summary of System Features
  - People want a short summary of the big ideas.
    - Listing the use case names is too detailed or low-level - there could be 30 or 50 use cases
    - Some interesting major features stakeholders really want to know is hidden in the use case names (e.g., payment authorization)
    - Some noteworthy features span or are orthogonal to the use cases. (e.g., "The system should be able to interact with existing third-party accounting, inventory, and tax calculation systems.")

121

Commentary: Vision 5

- For example:
  - Summary of System Features
    - sales capture
    - payment authorization (credit, debit, check)
    - system administration for users, security, code and constants tables, and so forth.
    - automatic offline sales processing when external components fail
    - real-time transactions, based on industry standards, with third-party systems, including inventory, accounting, human resources, tax calculators, and payment authorization services
    - definition and execution of customized "pluggable" business rules at fixed, common points in the processing scenarios
- Features should pass this linguistic test:
  - **The system does <feature X>.**
  - E.g. The system does payment authorization.

122

Commentary: Vision 5

- Question: how many system features should a Vision document contain?

Guideline

A Vision with less than 10 features is desirable – more can't be quickly grasped. If more, consider grouping and abstracting the features.

123

Commentary: Vision 5

- It is common to organize a two-level hierarchy of system features. For example:

  - Summary of System Features
    - POS services:
      - sales capture
      - payment authorization
      - ...
    - Inventory management:
      - automatic reordering
      - ...

Guideline

Terse is good in the Vision — indeed, in any document.

124

Commentary: Vision 6

- Guideline: should we duplicate other requirements in the Vision? (note: Vision summarizes other requirements)
  - Avoid duplication in both the Vision and Supplementary Specification (SS). Rather, record them only in the SS. In the Vision, direct the reader to the SS for the other requirements.
- Guideline: should you write the Vision or Use Cases first?
  - Working on one artifact influences and helps clarify another
  - A suggested sequence is:
    - Write a brief first draft of the Vision.
    - Identify user goals and the supporting use cases by name.
    - Write some use cases in detail, and start the Supplementary Specification.
    - Refine the Vision, summarizing information from these.

Loop

125

「依據測試結果，讀者大約讀到功能性需求的第37頁時就會睡著了，那我們把『待解決問題』這個小節放到第40頁就對了」

"Based on our tests, the business stakeholders fall asleep around page 37 of the Functional Requirements Specification. Put the Issues Section on page 40."

126

NextGen Example: A (Partial) Glossary 1 (2 pages)

- Revision History

Version	Date	Description	Author
Inception draft	Jan 10, 2031	First draft. To be refined primarily during elaboration.	Craig Larman

- Definition

Term	Definition and Information	Format	Validation Rules	Aliases
item	A product or service for sale			
payment authorization	Validation by an external payment authorization service that they will make or guarantee the payment to the seller.			

127

NextGen Example: A (Partial) Glossary 2

Term	Definition and Information	Format	Validation Rules	Aliases
payment authorization request	A composite of elements electronically sent to an authorization service, usually as a char array. Elements include: store ID, customer account number, amount, and timestamp.			
UPC	Numeric code that identifies a product. Usually symbolized with a bar code placed on products. See www.uc-council.org for details of format and validation.	12-digit code of several subparts.	Digit 12 is a check digit.	Universal Product Code
...	**...**			

128

Commentary: Glossary (Data Dictionary) 1

- The Glossary is a list of noteworthy terms and their definitions
- In the UP, the Glossary also plays the role of a data dictionary
  - A document that records the metadata of data.
- During inception the glossary should be a simple document of terms and descriptions.
- During elaboration, it may expand into a data dictionary. Term attributes could include:
  - aliases
  - description
  - format (type, length, unit)
  - relationships to other elements
  - range of values
  - validation rules

constitute requirements with implications on the behavior of the system

- Guideline: Start the Glossary early. It will quickly become a repository of detailed information.

129

NextGen Example: Business Rules (Domain Rules) 1 (2 pages)

- Revision History

Version	Date	Description	Author
Inception draft	Jan 10, 2031	First draft. To be refined primarily during elaboration.	Craig Larman

- Definition

ID	Rule	Changeability	Source
RULE 1	Signature required for credit payments.	Buyer "signature" will continue to be required, but within 2 years most of our customers want signature capture on a digital capture device, and within 5 years we expect there to be demand for support of the new unique digital code "signature" now supported by USA law.	The policy of virtually all credit authorization companies.

130

NextGen Example: Business Rules (Domain Rules) 2

ID	Rule	Changeability	Source
RULE2	Tax rules. Sales require added taxes. See government statutes for current details.	High. Tax laws change annually, at all government levels.	law
RULE3	Credit payment reversals may only be paid as a credit to the buyer's credit account, not as cash.	Low	credit authorization company policy

131

Evolutionary Requirements in Iterative Methods

Discipline	Artifact	Incep.	Elab.	Const.	Trans.
	Iteration	I1	E1..En	C1..Cn	T1..T2
Business Modeling	Domain Model		s		
Requirements	Use-Case Model	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
	Business Rules	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		
	Data Model		s	r	

132